Cahier d'exercices

Introduction aux Langages et Automates (R4.12)

C. Attiogbé, Nantes, Janvier 2024 (Version 1, maj 2026)



IUT de Nantes - BUT2 Informatique - 2025-2026

Résumé

— Sem1 : Mots - Langages (uniquement en TD, pas de programmation)
— Sem2 : Langages - Grammaires (uniquement en TD, pas de programmation)
— Sem3+ : Automates (TD et TP programmation en kotlin)

Des **évaluations** seront faites au fur et à mesure ; dans les séances de TD.
Vous devez toujours être prêt à rendre un/des exercice/s demandé/s.

Rappels des Apprentissages critiques ciblés par cet enseignement :
(voir votre Livret d'Etudiants)

— AC22.01 - Choisir des structures de données complexes adaptées au problème
— AC22.02 - Utiliser des techniques algorithmiques adaptées pour des problèmes complexes (par ex. recherche opérationnelle, méthodes arborescentes, optimisation globale, intelligence artificielle...)

Les exercices proposés visent donc à couvrir ces apprentissages critiques

AC22.01	✓	AC22.02	✓

Equipe pédagogique 2025-2026 :
Nicolas Hernandez - Christian Attiogbé (resp.)

Chapitre1 : Alphabet - mots - langages

Rappelons que les éléments ou symboles d'un alphabet peuvent être concaténés pour former des mots ; deux mots peuvent être concaténés pour former un autre mot ; la longueur d'un mot w se note $|w|$. Un mot w concaténé 2 fois à lui-même se note w^3.

Alphabet, mots

Dans la série d'exercices suivants, on n'utilisera que les opérations sur les ensembles (union, intersection, différence...) et les opérations sur les caractères et les mots (concaténation, longueur).

Exercice 1 Considérons l'alphabet $\mathcal{A} = \{0, 1\}$;

1. Construire sur $\mathcal{A}$ quelques mots.
2. Exprimer sur $\mathcal{A}$, en utilisant une notation ensembliste, l'ensemble M_n des mots de longueur n où n est un entier naturel.
3. Construire M l'ensemble des mots commençant par 1, suivis d'une suite quelconque et se terminant par 0.
4. Construire l'ensemble M des mots constitués d'une suite quelconque de 0 suivie d'une suite quelconque de 1.
5. Construire l'ensemble M des mots constitués d'une suite d'alternance de 0 et de 1. (ou encore de 1 et 0)
6. Construire M l'ensemble des mots contenant un seul 0
7. Construire sur $\mathcal{A}$ l'ensemble des mots contenant la lettre A [1].
8. Construire M l'ensemble des mots préfixés par le mot `111`
9. Construire M l'ensemble des mots préfixés par un mot constitué d'une répétition de `1` mais dont la longueur est un multiple (non nul) de 3 ; par exemple `111` ou `111111111`, ...

1. Le caractère ASCII A, en hexadécimal est 41

10. Construire l'ensemble M des mots contenant le mot 11011.
11. Construire toujours sur l'alphabet $\mathcal{A}$ l'ensemble M des mots contenant le(sous-)mot 'DONE'.
12. Pouvez-vous exprimer sur $\mathcal{A}$, en utilisant une notation ensembliste, l'ensemble des mots contenant autant de 0 que de 1 ? si oui comment et sinon pourquoi ?

Exercice 2 Considérons l'alphabet $\mathcal{A}_1 = \{do, re, fa, sol\}$ et l'alphabet $\mathcal{A}_2 = \{mi, si, la\}$;

1. Construire sur $\mathcal{A}_1$ quelques mots.
2. Construire sur $\mathcal{A}_2$ quelques mots.
3. Construire l'alphabet $\mathcal{A}_3 = \mathcal{A}_1 \cup \mathcal{A}_2$
4. Construire l'alphabet $\mathcal{A}_4 = \mathcal{A}_3 \setminus \mathcal{A}_1$
5. Construire l'alphabet $\mathcal{A}_5 = \mathcal{A}_3 \cap \mathcal{A}_1$
6. Construire sur $\mathcal{A}_3$ quelques mots.
7. Construire sur $\mathcal{A}_3$ l'ensemble M_4 des mots de longueur 4.

Exercice 3 : Immatriculation française (de véhicules) Les plaques d'immatriculation valables en France depuis 2009 utilisent le format AZ-009-AZ ; il est composé de 7 caractères **alphanumériques** répartis en trois groupes : deux lettres, trois chiffres et deux lettres, ces trois groupes sont séparés par des tirets (-).

1. Considérons les immatriculations fictives suivantes :
   TS-637-DE, GR-069-BZ, AV-298-FG.
   - Donner l'alphabet A permettant d'écrire ces immatriculations.
   - Pourquoi la seule donnée de cet alphabet ne suffit-elle pas pour écrire uniquement les immatriculations valides ?
   - En conséquence quelle solution peut-on employer pour n'écrire que les immatriculations voulues ?

Exercice 4
Soit $\mathcal{A}_l = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p,$
$q, r, s, t, u, v, w, x, y, z\}$ l'ensemble des 26 lettres de l'alphabet latin ;
soit l'ensemble de mots M_1 :
$M_1 = \{$Je, Bahia, travaille, tête, demain, work, she, eat, soleil, leçon$\}$;
soit $\mathbb{B}$ l'ensemble des booléens (vrai ou 1, faux ou 0) ;

1. Donnez sous forme d'une **fonction** convenablement nommée, de M_1 vers $\mathbb{B}$, et à l'aide d'**un diagramme sagittal** (Euler-Venn), les mots qui font partie ou non des mots formés sur l'alphabet $\mathcal{A}_l$?

2. Dites, en argumentant, si l'ensemble des mots **français** est une partie de $\mathcal{A}_l^*$
3. Peut-on étendre $\mathcal{A}_l$ pour que l'ensemble des mots français soit une partie de $\mathcal{A}_l^*$? Comment ?
4. Quelles contraintes a-t-on pour construire des phrases syntaxiquement correctes (en français) ? par exemple : travaille je demain

Exercice 5 - jj / mm / aaaa Considérons l'ensemble $\mathbb{N}$ des entiers naturels, l'ensemble $S = \{/\}$ et l'ensemble $\mathcal{A} = \{0, 1, 2, \cdots, 9\}$.

1. En prenant des exemples, montrer la différence entre les éléments de $\mathbb{N} \times S$, ceux de $\mathbb{N} \cup S$, ceux de $\mathcal{L}_{\mathbb{N}}.\mathcal{L}_S$, ceux de $\mathcal{L}_{\mathcal{A}}.\mathcal{L}_S$, ceux de $\mathcal{A}^*.S$
2. On veut décrire les mots de la forme $jj/mm/aaaaa$ ou où jj, mm et $aaaa$ sont des entiers naturels. Proposez une solution.

Exercice 5 bis Considérons l'ensemble $\mathbb{N}$ des entiers naturels et l'ensemble $S = \{/\}$.

1. En prenant des exemples, montrer la différence entre les éléments de $\mathbb{N} \times S$, ceux de $\mathbb{N} \cup S$ et ceux de $\mathcal{L}_{\mathbb{N}}.\mathcal{L}_S$.
2. On veut décrire les mots de la forme $n \uparrow m$ ou $n \downarrow m$ où n et m sont des entiers. Proposer une solution.

Exercice 6 (facultatif)

1. Considérons l'alphabet $\mathcal{G}_1 = \{<, >, \vee, \wedge\}$
   On veut se servir de cet alphabet pour donner des instructions pour faire des mouvements cycliques de déplacements (gauche/droite, bas/-haut) indéfinis (mais en tournant dans le sens des aiguilles d'une montre, toujours comme ceci ↻).
   Un mouvement cyclique se fait par les instructions droite ($>$) , bas ($\vee$), gauche ($<$), haut ($\wedge$). Chaque instruction est présente au moins une fois, mais on peut répéter plusieurs fois chaque instruction.
   — Donner l'ensemble I des mots représentant des instructions correctes (pour des ronds bien faits selon les hypothèses ci-dessus)
2. Considérons l'alphabet $\mathcal{G}_2 = \{\square, \triangle, \bullet, \triangledown, \bowtie\}$; Parmi les mots suivants, lesquels ne sont pas formés sur $\mathcal{G}_2$?
   (a) $\triangledown\triangledown\square\triangle$
   (b) $\square \bullet \bullet\triangleleft$
   (c) $\triangleleft\triangleleft\triangleright\triangleleft\bullet \bowtie \triangledown$
   (d) $\triangledown\triangledown\blacksquare\square\triangle$
   (e) $\square\triangle\square\triangledown\bullet\bullet$
   (f) $\square\square\square\blacktriangle$

Chapitre2 : Langages - grammaires

Langages et grammaires

Rappel de cours

À l'image des langages naturels, les langages formels utilisent des règles pour la structuration des phrases : on parle de grammaires.
Le langage est un outil qui véhicule des expressions, des idées, des concepts, des objets, ...
Un **ensemble de mots** construit sur un alphabet, **forme un langage**.

Une grammaire engendre un langage (contraint).

Toutes les suites de mots (phrases) construites en respectant les règles de la grammaire d'un langage n'ont pas forcément de sens. On peut exprimer des suites de mots (phrases) sans tête ni queue ; par exemple : le berger gerbé a mergé marchant en !

On peut aussi exprimer une même idée de différentes façons dans un même langage.

Contrairement au langage naturel, il n'y a à priori pas d'ambigüité dans le langage formel.
Nous traitons dans ce cours de **langages formels** (comme les langages de programmation).

Exercice 1 : rappels sur les grammaires

1. Formuler à votre façon, mais justement, la définition d'une grammaire.
2. Quel est est le lien entre une grammaire et un langage ?

3. Soit la grammaire G_1 suivante :

$$
\begin{array}{lll}
G_1 = & \langle & \\
 & T = & \{x,\ ;,\ -,\ =,\ +,\ *,\ 2,\ 7\}, \\
 & N = & \{V, E, P, I, OP, SI\}, \\
 & S = & P, \\
 & R = & \{P \rightarrow\ I\ \ SI \\
 & & \ SI\ \rightarrow\ ;\ I\ SI \\
 & & \ SI \rightarrow\ \epsilon \\
 & & \ I\ \rightarrow\ V = V\ \ OP\ \ E \\
 & & \ V\ \rightarrow x \\
 & & \ OP\ \rightarrow\ +\ |\ -\ |\ * \\
 & & \ E\ \rightarrow\ 2\ \ |\ \ 7\ \ |\ \ V\ \ OP\ \ E \\
 & & \} \\
 & \rangle &
\end{array}
$$

(a) Donner les **terminaux** de G_1 ?

(b) Donner l'**axiome** de G_1 ?

(c) Donner deux exemples de **règles de production** de G_1.

(d) Qu'est-ce qu'une **dérivation** ? comment la note-t-on ?

(e) Proposer, à partir de l'axiome de G_1, une suite de trois(3) dérivations (notée $\cdots \Rightarrow^3 \ldots$).

(f) Qu'est-ce qu'une **forme terminale** et avez-vous obtenu une forme terminale dans la question précédente ?

(g) proposer sous-forme d'un arbre (inversé), une dérivation multiple de G_1, jusqu'à obtenir une forme terminale.

Exercice 2 : analyse empirique

Soit le "programme" suivant, où les espaces et les sauts à la ligne sont artificiels afin de faciliter la lecture :

```
x = x * 2 ;
x = x - x - x + 7
```

En considérant la grammaire G_1 de l'exercice précédent,

1. Proposer une suite de dérivations permettant de partir de l'axiome pour analyser le "programme"
2. Proposer la suite de "correspondances/réductions" (opération inverse d'une dérivation) permettant de partir du "programme" et de le reconnaître comme un mot du langage engendré par la grammaire.
3. Comparer les formes arborescentes obtenues.

Exercice 3 : les expressions arithmétiques et les instructions

1. On considère des **expressions** arithmétiques simples sans les parenthèses ; par exemple : x+y-4 ou 2x*y+1 ou 3x-4y+1-x
   Donner la grammaire permettant d'écrire ces expressions arithmétiques.
2. Soient des **instructions** simples de la forme Var := ExprArithm
   Elaborer la grammaire permettant d'écrire ces instructions simples. Ici Var désigne une variable ; on prendra juste deux valeurs de variables pour illustrer (x et y).

Exercice 4 : bolang

Soit bolang un langage dans lequel les signatures (ou interfaces) des fonctions ont la forme suivante : [Feature]* nomfonction [(list de paramètres typés)]

[attribut] : indique que attribut est optionnel
[attribut]* indique que attribut est optionnel, et peut se répéter.

Exemples de signatures de fonctions dans bolang :

void afficher (type prm)
public void trierListe ()
void trier (type uneListe)
static void static void void fusionner(type liste1 ; type liste2 ; type liste3)

On se restreint dans l'exercice à la liste de Feature suivante : public, static, void, synchronized, abstract, final et aux valeurs de type suivantes : int, ListInt.

prm ou nomfonction sont des suites de caractères alphabétiques.
liste$_i$ est une liste de paramètres (prm) séparés par des " ;".

1. Formaliser à l'aide d'une grammaire la signature d'une fonction du langage bolang.

[Exercice 6 :] grammaire et découverte de la notation BNF
Soit la grammaire incomplète G suivante exprimée en notation Backus and Naur Form (BNF). Dans cette notation le symbole $\rightarrow$ est remplacé par `::=`.

```
  Pgm    ::= Header Block
  Header ::= Name '(' ArgList ')'
  Name   ::=  'f' | 'g'
  Arg    ::= Var ':' Type
  ArgList::= Arg  | Arg ';' ArgList
  Var    ::= 'x' | 'y'
  Type   ::= 'Num'
  Block  ::= 'begin' StatmentList 'end'
  StatmentList ::= Statment   | Statement ';' StatementList
  Statement ::= Variable ':=' Expr
  /*  expressions élémentaires  */
  Expr ::= Number  | Expr Binop Expr   | Unop Expr
  Binop ::= '+' | '-' | '*' | '/'
  Unop  ::= '-'
  Number ::= 0|1|2|3|4....
 }
```

1. Enumérer les éléments du vocabulaire terminal,
2. Enumérer les éléments du vocabulaire non-terminal,
3. Quel est l'axiome de G ?
4. A quoi servent les règles de production d'une grammaire ?
5. Donner un exemple de règle de production de G.
6. Reécriver en conséquence la grammaire sous la forme suivante :

   ```
   G = {T, N, S, R}
   T = { ??? }
   N = {Pgm, ??? }
   S = ???
   R = { ??? }
   ```

7. Donner un exemple de mots (programme) correct par rapport à ce langage.

Exercice 7 : la francette

On considère le langage francette, dans lequel la règle de structuration des suites de mots correctes est : Mot^+ Verbe Mot^*.

On donne comme Mot : le, la, chat, bouteille
On donne comme Verbe : avale, mange

Toute ressemblance entre francette et le langage français est purement fortuite ; on ne fait aucune hypothèse sur le sens habituel des mots en français.

Par exemple les suites de mots suivantes sont correctes en francette :

la chat avale bouteille
le bouteille la mange chat
chat avale

1. Proposer une Grammaire permettant de construire des phrases syntaxiquement correctes en francette
2. On veut maintenant contraindre la grammaire pour que les suites de mots correctes soient de la forme suivante :

```
la chat avale la bouteille

le chat mange la bouteille

le bouteille mange la chat
...
```

   Proposer une nouvelle grammaire G_2.
3. Quel est le type de cette grammaire G_2

Exercice 8 : langage, grammaire et type
Soit un alphabet $A = \{1, 0\}$.

1. Construire **le langage** L constitué des entiers naturels impairs.
2. Construire **une grammaire** G qui engendre L.
3. Quel est **le type** de votre grammaire G ?
4. Même question avec les nombres divisibles par 2.

Exercice 9 : langage, grammaire et type
Soit un alphabet $A = \{a, b\}$ et le langage $L = \{ab^n a \mid n \in \mathbb{N}\}$

1. Construire une grammaire G qui engendre L.
2. Quel est le type de votre grammaire G ?

Exercice 10 : construire un langage adhoc

Il arrive souvent, que l'on définisse un petit langage adhoc ou spécifique à un domaine, dans le cadre d'une application logicielle. On appelle ces langages des langages spécifiques à un domaine ou *Domain-specific language* (DSL).

Construisons L_1, un petit DSL pour écrire des pseudo-algorithmes comme celui-ci.

```
Z <-- X;
Y <-- 1;
WHILE not(Z = 0) DO
Y <-- mult (Y, Z) ;  // multiplier Y et Z
Z <-- subs(Z, 1)     // soustraire 1 de Z
END
```

1. Ecrire une grammaire (de type 3 pour) qui engendre le langage L_1.

Exercice 11 : analyseurs lexical et syntaxique - Compilation

Les compilateurs de langages de programmation (classiques) ont une architecture précise comme celle indiquée dans la figure .

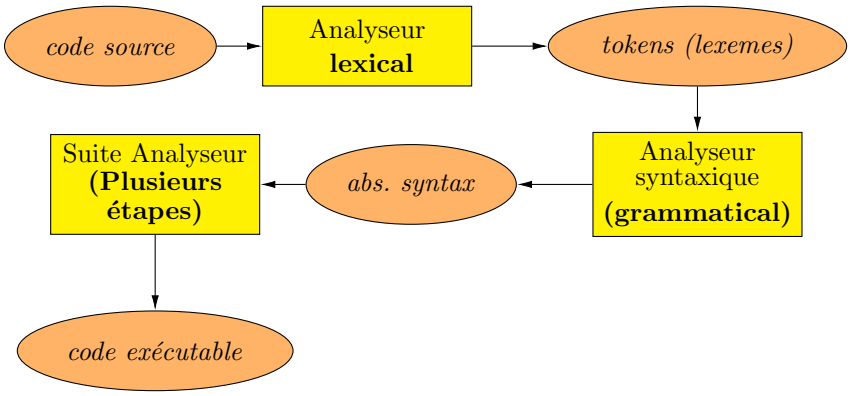


FIGURE 1 – Schéma du processus de compilation

L'analyseur lexical construit/identifie des mots à partir des caractères d'un alphabet, l'espace est utilisé comme séparateur ; ces mots sont appelés des *tokens* ou lexèmes. Les suites de tokens/lexèmes issues de l'analyse lexicale sont données en entrée à l'analyseur syntaxique (grammatical) qui produit un arbre de syntaxe abstraite lorsque les suites de lexèmes respectent les règles de la grammaire (sinon il y a erreur/s de syntaxe). L'arbre de syntaxe abstraite est ensuite utilisée/transformée à travers plusieurs étapes afin d'aboutir à un code intermédiaire puis un code exécutable correspondant au code source initiale.

Lorsque on a lu 'x', on a reconnu en fait un lexème qui sera traité comme VAR. Lorsqu'on lit '+' ou '*' ou '-' on reconnaît un lexème qui est traité comme

un OP dans la grammaire, ainsi de proche en proche on analyse les formes de la grammaire. On peut retourner ('x', VAR) ou ('*', OP), etc à l'analyseur grammatical.

En reprenant les exemples de l'**exercice 2**,

1. donner dans l'ordre, la suite de lexèmes envoyés à l'analyseur syntaxique par l'analyseur lexical.
2. donner l'arbre de syntaxe abstraite construite par l'analyseur syntaxique.

Chapitre 3 : Automates

Automates déterministes

Rappel de cours

Les exercices 1 et 2 servent de principaux rappels. Vous devez travailler ici avec vos notes de cours, les supports du cours et le poly du cours.

Consignes

Vous aurez besoin de vos réponses/solutions des exercices pour programmer lors des séances suivantes. Il faut donc **travailler, décrire et noter rigoureusement vos automates**.
Cette série d'exercices aboutira en plus du/des contrôle/s continus, à une programmation d'analyseurs. **Votre réalisation sera rendue le vendredi de la dernière semaine de TD/TP (16 février 2024)** ; elle sera notée.

Préparation des TPs (+ évaluation)

Les exercices qui vont suivre vous permettront de préparer les réalisations que vous aller faire en TP (en Kotlin). En TD, nous allons construire plusieurs automates, qui serviront en entrée d'une application que vous allez construire. Le principe est donné dans la figure 2

La figure 9 donne l'architecture globale de ce qu'on vous demandera de faire en TP. Il y aura plus de détails plus tard.



FIGURE 2 – Architecture simplifiée de l'application à écrire

Anticipation A l'issu des TD/TP, vous proposerez à l'utilisateur de choisir un analyseur (donc un automate spécifique) avant de lui demander la chaîne (mot) à analyser ; en fonction de son choix, vous chargerez un de vos automates (chacun créé par une méthode dédiée), puis vous analyserez la chaîne donnée (lue en entrée) et indiquerez si la chaîne est correcte (OK) ou incorrecte (KO). Voici ce à quoi cela va ressembler.

```
---------- Nantes Université, BUT2 Informatique,  2024 ------------
------ TP - Modélisation et analyse à l'aide des automates   ------
-------  Menu de l'application (développée par par Moa TOA) -------
    1. Smiley (pour reconnaître un des smileys)
    2. HH:MM  (pour reconnaître une heure bien formée)
    3. JJ/MM/AAAA
    4. Adresses électroniques
    5. Polynômes
    6. Expressions arithmétiques
    7. Polynômes
    ...
    99. Arrêt de l'application
Votre choix (1-99) ?
Vous saisirez ensuite la chaîne à analyser, Merci
-----------------------------------------------------------------
```

Vous trouverez en page 24, des indications complémentaires pour les travaux pratiques.

Exercice 1 - rappels (répondre en relisant votre cours)

Soit l'automate A_1 de la figure Fig. 3.

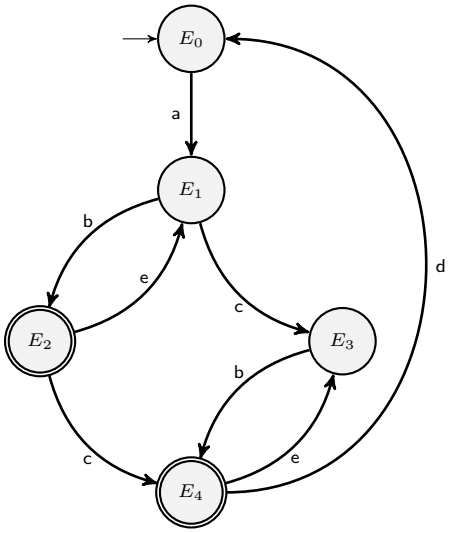


FIGURE 3 – Automate A_1

1. Combien d'états possède l'automate A_1 (lister les) ?
2. Donner et les états initiaux et l'ensemble des états finaux de A_1
3. L'automate A_1 est-il déterministe ? pourquoi ?
4. Donner δ (la relation de transition) sous la forme d'une fonction (en énumérant ses éléments/couples)
5. Dans quel état se trouve l'automate A_1 après avoir lu le mot `bbabb` ?
6. Dans quel état se trouve l'automate A_1 après avoir lu le mot `abebec` ?
7. Qu'est-ce qu'un automate accepteur ?
8. Donner un ou deux autres classes/types d'automates déterministes (vus en cours)
9. Ce mot (`abebec`) est-il reconnu / accepté par l'automate ? pourquoi ?
10. Mêmes questions pour les mots `abebebecbeb` et `abcdabcebdacb`

Exercice 2 : mots et automates accepteurs

Soit l'automate A_2 présenté en Fig. 4

1. Donner 5 mots différents reconnus par cet automate (dont au moins un de longueur ≥ 6).

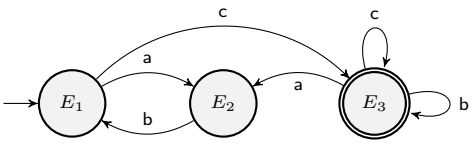


FIGURE 4 – Un automate (A_2)

2. Donner 3 mots différents non reconnus par cet automate.
3. En reprenant dans votre cours la définition formelle d'un automate ($\langle, S, A, \delta, S_0, S_f\rangle$), donnez en extension, la fonction de transition δ de l'automate A_2, (c'est-à-dire sous la forme d'un ensemble de couples).
4. Quelle est la suite des états par lesquels l'automate passe en lisant le mot `abcbbc` ?

Modélisation (construction d'automates) - Analyse

Exercice 3 : Modélisation avec un automate - smileys

Prenez une liste de *smileys* communément utilisés (en forme textuelle = suite de caractères).
Ecrivez un automate qui reconnaît exactement les *smileys* de votre liste (et rien que ceux là).

Exercice 4 : Analyse : entiers paires, impaires

En considérant l'alphabet $\{0, 1\}$

1. Ecrire un automate qui permet de reconnaître des entiers pairs non nuls.
2. Ecrire un automate qui permet de reconnaître des entiers impairs.

Exercice 5 : Analyseur hhmm - introduction du TP

Etant donné l'automate A_c de la figure Fig.5

1. Donner plusieurs mots différents acceptés par A_c
2. En déduire la spécification modélisée par A_c (= ce que fait A_c).
3. En précisant les principaux constituants de l'automate ($\langle S, A, \delta, S_0, S_f\rangle$), décrivez l'automate sous forme d'une table (la fonction de transition). Pour vous guider on donne le début de la table :

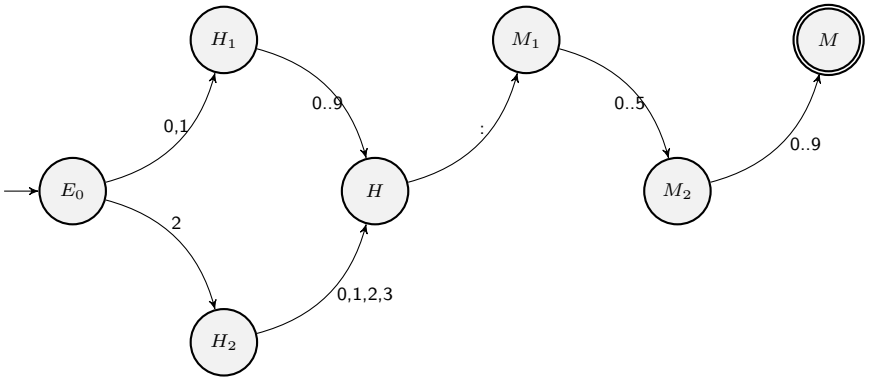


FIGURE 5 – Un automate accepteur

```
VARIABLES
S = ...
A = ...
delta =  /* table ----> */
$S_0$ = ...
$S_f$ = ...
```

$S \backslash A$	0	1	2	$\cdots$
E_0	H_1	H_1	H_2	$\cdots$
H_1	H	H	H	$\cdots$
H_2	H	H	H	$\cdots$
H	?	?	?	$\cdots$
$\cdots$	$\cdots$	$\cdots$	$\cdots$	$\cdots$

4. Ecrivez un **pseudo-algorithme** qui donne le comportement de l'automate, pour une suite de caractères en entrée. Pour vous guider, voici un algorithme (incomplet, à parfaire en TP) :

Algorithm 1 Implantation du comportement d'automate

$Etatc \leftarrow E_0$ {partons de l'état initial}
{On a une suite de caractères en entrée}
{On prendra carCourant comme le caractère suivant dans la suite}
while $Etatc \neq E_f \wedge Etatc \neq EtatNonDef$ **do**
  lire carCourant {on parcourt la suite de caractère en entrée}
  afficher Etatc, carCourant
  $Etatc \leftarrow \ \delta(Etatc, carCourant)$
end while
if $Etatc = E_f$ **then**
  Bien terminé : la suite de caractères en entrée est reconnue
else {On s'est arreté sur un état non défini}
  Echec : la suite de caractères n'est pas reconnue
end if

5. Proposez une bonne façon de traiter les cas d'erreur, par exemple dans un état courant, que faire si on lit un caractère inattendu ? (`18:72`, `31:04`, `08;10`, ...)

6. Modifiez l'automate en un automate de Mealy où l'action de chaque transition consiste à afficher le caractère lu.

Exercice 6 : entiers multiples de 3, multiples de 100, ...

On considère l'alphabet constitué des chiffres $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

1. Ecrire quelques chiffres et nombres, entiers naturels (une vingtaine au moins) qui sont des multiples de 3.
2. Observer et lister les régularités que remarquées dans l'expression de ces nombres.
3. En conséquence, construire un automate qui reconnaît/accepte les entiers multiples de 3.
4. Préparer la table de transition pour votre TP.
5. Construire l'automate qui accepte les nombres entiers qui sont multiples de 100.

Intermède : encodage des automates déterministes

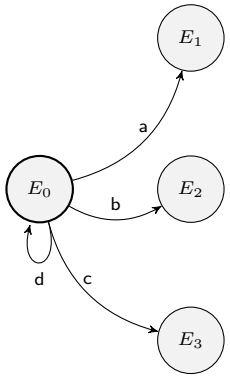


Figure 6 – Un noeud d'un automate ...

Une façon d'encoder un automate A est de construire une structure manipulable fidèlement comme le quintuplet qui définit formellement l'automate ; en plus du nom de l'automate, il faut ses 5 constituants. Dans une approche Classe/Objet, il faudra des *getter/setter* pour chaque constituant. Il n'est pas judicieux de représenter la fonction de transition par un tableau/matrice.

— une liste des états, (où chaque état a un nom et des attributs),
— l'état initial,
— les états finaux,
— les transitions de l'automate sont l'union des transitions de ses états.

— des méthodes pour exploiter l'automate.

Chaque état (une classe) a parmi ses attributs, les transitions sortantes et quel/s symbole/s de l'alphabet permettent d'atteindre ses transitions. Les transitions sortantes sont par exemple représentées par une HashMap < clé, valeur>, où la clé est un symbole et la valeur l'état atteint.

Exemple : l'état E_0 de la figure 6 aura comme attribut la *hashmap* outgoingTransition suivante : $\{\langle d, E_0\rangle, \langle a, E_1\rangle, \langle b, E_2\rangle, \langle c, E_3\rangle\}$

Ainsi, dans chaque état, on définit facilement une fonction qui décide si un symbole s lu est une action sortante ou pas (ss est parmi les clés) ; et lorsque c'est le cas, une fonction qui donne immédiatement l'état atteint : outgoingTransition(ss).

Exercice 7 : Analyseur jjmmaaaa

Dans le formulaire d'un logiciel, les utilisateurs doivent saisir dans une zone de texte une date sous la forme jj/mm/aaaa. Pour simplifier ici, on ne traîte pas les années bissextiles (on accepte donc les années de **0001 à 9999**, et prenons 28 jours pour le mois de **février**). Les jours vont de 1 à 28, 30, 31 selon les mois. Les mois vont de 1 (janvier) à 12 (décembre).

Dans une autre zone de texte dans le formulaire, les utilisateurs doivent saisir une heure sous la forme hh:mm:ss

1. Ecrivez un automate qui reconnaît une expression de date bien formée (jj/mm/aaaa).
2. Ecrivez un automate qui reconnaît une expression d'heure bien formée (hh:mm:ss).
3. Pensez à la phase d'analyse du logiciel qui traite le formulaire ; que peut-on réutiliser par rapport à la reconnaissance des expressions ?

Exercice 8 : Analyseur des adresses électroniques

Une adresse électronique normalisée est sous la forme `Prenom.Nom@nomDeDomaine`, ou `chaineQuelconque@nomDeDomaine`.

Les caractères alphanumériques et quelques caractères spéciaux sont autorisés. Par exemple `paul-annie.martin44@univ-nantes.fr` ou `karima-dong.Yamashi@etu.univ-nantes.fr` sont des adresses correctes.

1. Ecrivez un automate qui reconnaît une adresse mail bien formée.
   Procédez pas à pas, en commençant par des hypothèses simples puis en augmentant vos hypothèses.
2. Modifiez l'automate pour reconnaître une liste d'adresses électroniques séparées par des " ;"

Exercice 9 : Analyseur de polynômes

Construisez un automate qui reconnaît l'expression d'un polynôme sous la forme : $aX^{**}2 \oplus bX \oplus c$ où a, b et c sont des entiers (entre 1 et 25).
Les espaces en nombre quelconque sont tolérés ; X est la seule variable autorisée ; $\oplus$ représente les deux opérateurs `-` ou `+`.
Par exemple
`8X**2 - 15X + 3` ou bien `4X**2 + 3X - 9` sont des polynômes correctement écrits,
alors que `42X**2 + 7X` ou bien `4X*2 -3X + 9` sont syntaxiquement incorrects.

Exercice 10 : analyse des instructions de programmes

On considère des instructions d'affectation simples, parenthèsées, de la forme :
`x := <EXPRAR>`
où `<EXPRAR>` représente les expressions arithmétiques correctes simples, sans imbrication de parenthèses, comme par exemple :
`(x)` `(y)` `(x + y)` `(x - y)` `(x + y + x - y / x + y)`

1. Proposez un automate déterministe permettant de reconnaître une instruction d'affectation avec une expression arithmétique bien parenthésée (`<EXPRAR>`), mais sans imbrication de parenthèses, utilisant uniquement les variables x, y et les opérateurs +, - et / .

2. Comment peut-on modifier l'automate proposé afin de reconnaitre une suite d'affectations avec le caractère **;** comme séparateur ?

Exercice 11 : simulation de visites virtuelles

Soit l'automate A_m suivant (Figure 7) qui donne la position d'un visiteur dans une maison. Le visiteur vient de l'extérieur et peut parcourir des pièces de la maison en franchissant des portes.

Nous voulons raisonner sur le plan de la maison, en partant des parcours possibles ou non, des pièces.

Nous voulons ici utiliser les chemins ou les séquences d'états pour expliciter les questions, pour les poser ou pour y répondre.

1. Peut-on aller dans la chambre sans passer par le séjour ?
2. Est-ce que toutes les chambres sont accessibles à partir du SAS ?
3. Est-ce qu'un WC est-accessible à partir du SAS ?
4. Nous considérons que les états représentent les pièces ou endroits de la maison (chambre, wc, terrasse, ...) où se trouve un visiteur à un moment donné ;

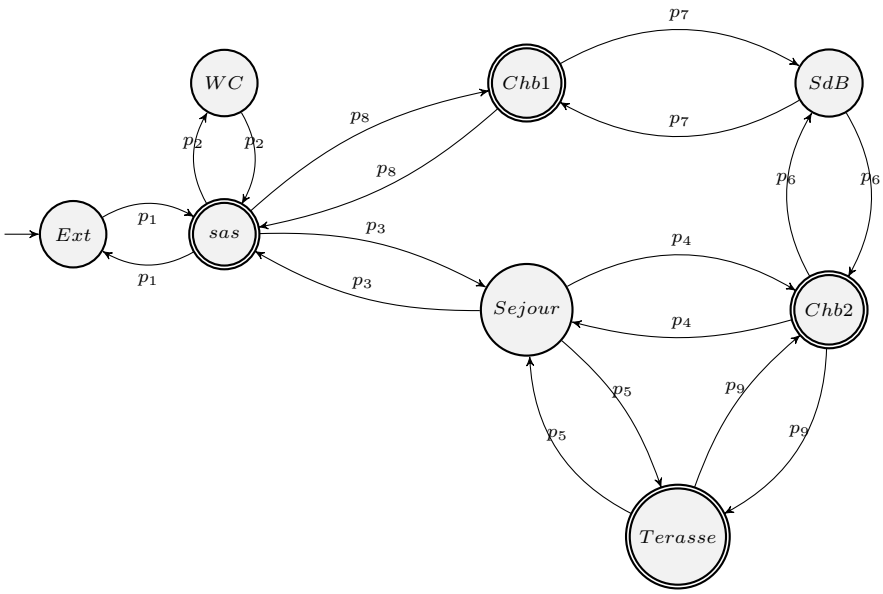


FIGURE 7 – Automate A_m du visiteur

les transitions représentent les portes d'accès d'un endroit à un autre.

Proposer un dessin sommaire du plan de la maison.

Exercices d'ouverture - pour votre curiosité

Les automates comme nous l'avons vu en cours sont des objets de modélisation très utilisés, et adaptés dans plusieurs domaines.

- Automates avec des temps (durées, ou intervalles de durée) sur des transitions.
- Automates avec des probabilités sur les transitions ; une transition est probable, avec la valeur associée, sachant que la somme des transitions sortant d'un état est 1.
- Chaînes de MARKOV : un automate avec des transitions probabilisées et, une transition (probabilité de transition) vers un état ne dépend que de l'état courant.

Expression régulière

Exercice 20 : expression régulière et automate non-déterministe

Soit l'expression suivante :

$$a(\ (ab)^*cb^*c^*\ )^*\ +\ a((ba)^2cb^*)^*$$

1. Proposer un automate non-déterministe qui correspond à cette expression régulière
2. Peut-on déterminiser cet automate (pourquoi) ?

Exercice 21 : Expression régulière : modélisation

Nous considérons après des abstractions lors d'une phase d'analyse dans un projet logiciel, une **fonctionnalité** du projet qui est **modélisée par un automate**.

Soit l'alphabet $A = \{a, b, c, d, e, f\}$ qui représente les abstractions des actions à faire dans la fonctionnalité considérée, et soit l'automate de la figure Fig. 8 qui modélise la fonctionnalité.

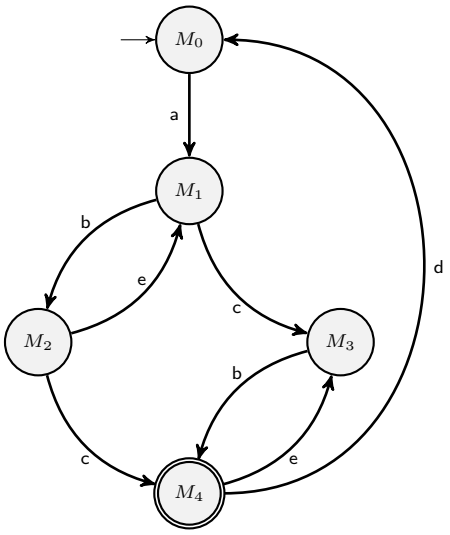


FIGURE 8 – Un automate accepteur

1. Ecrire une expression régulière équivalente à l'automate.
2. En considérant que le sens formel des opérateurs réguliers (c'est à dire : * représente une itération ;

+ représente un choix/alternative,
• représente une séquence), générer un squelette d'algorithme/programme correspondant à votre expression régulière.

3. Quelle conclusion pouvez-vous tirer à propos de la modélisation par les automates et de la construction des programmes ?

Exercice 22 : expressions régulières dans les langages de commande

Les expressions régulières sont utilisées par exemple dans la commande grep de Linux (Unix) pour chercher des lignes contenant un motif particulier dans un fichier : `egrep 'regexp' nomFicher`. Voir le manuel de `egrep` (man egrep).

`'regexp'` est l'expression régulière qui définit le motif cherché. La syntaxe des expressions régulières reprend les principaux opérateurs (* : le caractère/élément le précédent est répété 0 ou plusieurs fois ; + : le caractère/élément le précédent est répété 0 ou plusieurs fois ; — représente le choix. De plus `?` représente un caractère quelconque ; les parenthèses `(` et `)` entourent une expression régulière composée ; `[a-z]` les 26 lettres minuscules ; `[A-Z]` les majuscules ; `[0-9]` les chiffres de 0 à 9.

1. expliquer chacune des expressions régulières suivantes :

```
??remi*
??re(mi)+
do*(fa)+*do
(of)|(mi)+
(do)*.*s??m
```

2. mise en pratique ; soit un fichier testGrep.data contenant les lignes suivantes

```
doesolmi
doremifafaredodo afido
whiledododofasilmimisol
ifofendmirador
```

Donner le résultat de la commande `egrep 'regexp' nomFicher` avec chacune des expressions puis testGrep.data en argument.

TP : ‘Automates déterministes analyseurs-accepteurs

Dérivation de programmes analyseurs, en kotlin

En vous appuyant sur le pseudo algorithme étudié, et des indications ci-après (et www du module)

- Dérivez le programme kotlin correspondant à la reconnaissance d'une expression.
- Modifiez simplement l'automate en entrée du programme et testez avec le même programme, pour tous les exercice.

Implantation en Kotlin

Dérivez à partir de la modélisation sous forme d'automate et du pseudo-algorithme du comportement un programme/module/procédure (en Kotlin) qui donne la fonctionnalité modélisée dans les exercices précédents.
Nous vous conseillons de construire au moins 3 classes :

- **une classe Etat**,
- **une classe Automate** et
- **une classe principale**, MonApplication par exemple.

La classe Automate **doit contenir le 5-uplet** qui définit un automate.
On traitera les chaînes en entrée caractère par caractère.
La figure 9 donne l'architecture globale de ce qu'on vous demande de faire.



FIGURE 9 – Architecture simplifiée de l'application à écrire

Aussi vous adopterez une démarche combinant une *approche descendante* (allant de l'application vers les classes dont elle a besoin) et une *approche ascendante* (allant des classes et méthodes élémentaires vers leur utilisation dans l'application).

Aide à la conception. Pour implanter les automates en Java et notamment les transitions entre états, nous vous conseillons d'éviter les structures de données rigides ou inefficaces (par exemple, il faut concevoir la table de transitions autrement).

Remarquer que pour tout état de l'automate (sauf peut-être pour certains finaux), on a des transitions sortantes (allant vers d'autres états). Si on décrit toutes les transitions sortantes de chaque état, (et si on le fait pour tous les états) alors on a l'ensemble des transitions de l'automate. Par exemple, de l'état `E0` de la figure 5, on a trois transitions sortantes avec les actions `0, 1, 2`. Il s'agit de `(E0, 0, H1), (E0, 1, H1), (E0, 2, H2)`.

Etant dans `E0` l'action 0 amène à l'état `H1`, `2` amène à l'etat `H2`, etc. On va donc définir dans la classe Etat, les transitions sortantes (d'un état courant), avec une `HashMap<key, value>`. Ceci facilite grandement la manipulation de *delta*, l'écriture et la généralisation de votre application.

Aide à la mise au point. Utiliser une méthode Kotlin pour créer/charger votre automate. Cette méthode est appelée à partir de votre `main`. Une fois que vous aurez mise en œuvre votre première version de l'application, vous allez structurer le `main` à l'aide d'un simple menu alphanumérique (qui sera progressif) dans votre TP. Vous proposerez à l'utilisateur de choisir un analyseur avant de lui demander la chaîne à analyser ; en fonction de son choix, vous chargerez un de vos automates (chacun créé par une méthode dédiée), puis vous analyserez la chaîne donnée (lue en entrée).

```
--------------- Menu de mon TP -------------------------
1. Smiley (pour reconnaître un des smileys)
2. HH:MM  (pour reconnaître une heure bien formée)
3. JJ/MM/AAAA
4. Adresses électroniques
5. Polynômes
6. Expressions arithmétiques
7. Polynômes
...
99. Arrêt de l'application
Votre choix (1-99) ?
Vous saisirez ensuite la chaîne à analyser, Merci
------------------------------------------------------
```

Expérimentations (et pour aller plus loin) Pour faciliter la construction des automates, après avoir construit le(s) premier(s) directement dans le source de votre programme, vous pouvez systématiser la construction d'un automate à partir d'une représentation textuelle contenue dans un fichier texte.

Par exemple, on peut représenter chaque transition d'un automate sur une ligne du fichier texte : `etatD, etiquette, etatF`, pour construire la transition entre l'etat `etatD` et l'état `etatF`, avec le symbole `etiquette`.

Ainsi, en lisant ligne par ligne ce fichier, on peut construire progressivement en appelant une méthode dédiée à la construction d'une transition l'automate.

Pour cela, vous pouvez écrire un programme utilitaire (analyseur) comme sur la figure Fig. 10, qui lit ligne par ligne un fichier .txt et crée un automate à vide, puis lui ajoute des états, puis lui ajoute des transitions. Ce programme ne fait que quelques lignes !

Il faut aussi concevoir comment indiquer les états initiaux et finaux dans votre fichier texte en entrée.

Voici un schéma synthétique qui peut vous aider dans votre travail (Fig. 10).

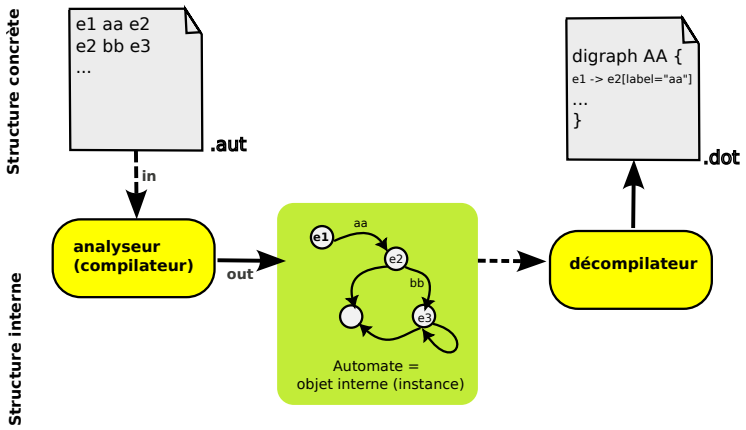


FIGURE 10 – Architecture simplifiée de compilation-décompilation

Enfin si vous voulez aller encore un peu plus loin, en très peu de lignes, vous pouvez par exemple dans une fonction *toString()* dédiée, générer un fichier en format graphique (.dot) de vos automates. Ainsi on peut visualiser les automates que vous avez analysés très simplement (commande dotty, etc voir la doc pour graphviz).

Aide à la mise au point en kotlin

- Utilisation des *map* {<clé, valeur>} en kotlin : `https://stackoverflow.com/questions/47204146/how-to-iterate-over-hashmap-in-kotlin`
- Lecture d'un fichier de texte ligne par ligne : `https://stackoverflow.com/questions/55182578/how-to-read-plain-text-file-in-kotlin`
- découpage (split) s'une ligne de texte en ces parties : `https://stackoverflow.com/questions/46038476/how-could-i-split-a-string-into-an-array-in-kotlin`
-

Documentation complémentaire

- Compiler un fichier .dot en .pdf `https://graphviz.org/docs/outputs/pdf/`
- dot/graphviz : `https://graphviz.org/Gallery/directed/`
- dot, automate `https://graphviz.org/Gallery/directed/fsm.html`